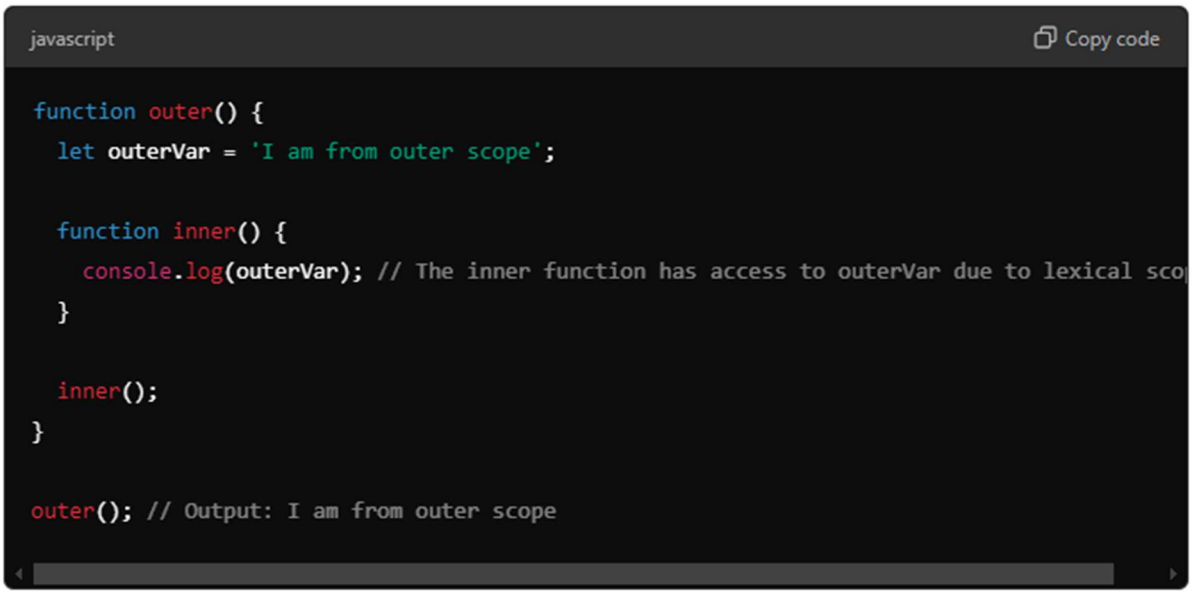
Lexical Scope

Lexical scope in JavaScript refers to the fact that the scope of variables is determined by their location in the source code, and nested functions have access to variables declared in their outer (parent) functions.

In other words, a function can access variables from the scope in which it was defined, not where it was called.

Example:




```
function outer() {
  let outerVar = 'I am from outer scope';

  function inner() {
    console.log(outerVar); // The inner function has access to
                           // outerVar due to lexical scope
  }

  inner();
}

outer(); // Output: I am from outer scope
```

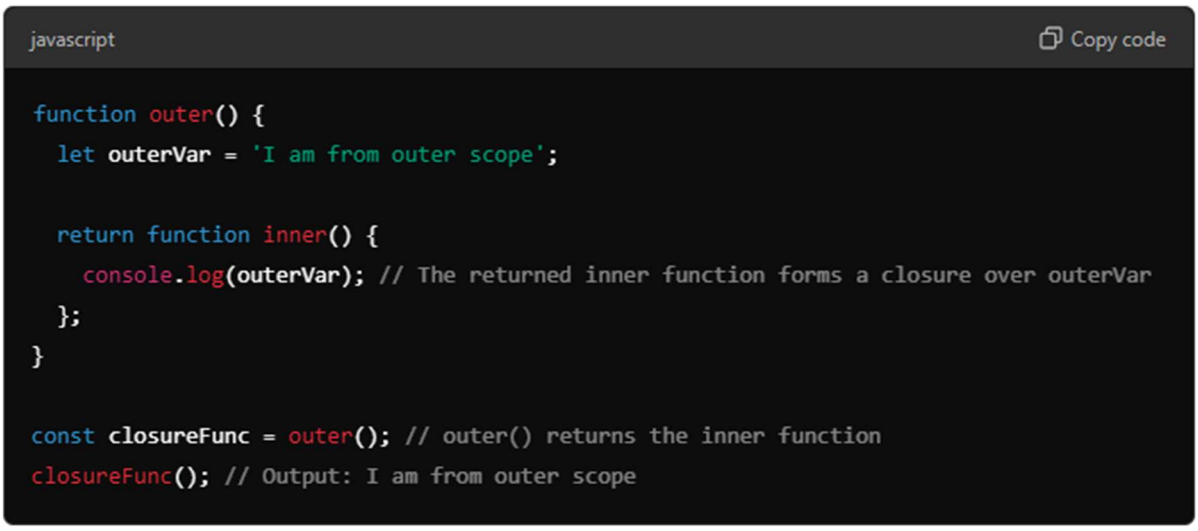
In this example, the `inner` function can access the `outerVar` variable because of lexical scoping. Even though `outerVar` is declared in the outer function (`outer`), it's still accessible within the `inner` function.

Closure

A **closure** is created when a function "**remembers**" its lexical environment, **even after** the function is executed and its outer function has finished running. It allows a function to access and manipulate variables from an outer scope even after the outer function has returned.

Closures "close over" variables from their parent scope, keeping them alive so that they can still be accessed later.

Example:




```
function outer() {
  let outerVar = 'I am from outer scope';

  return function inner() {
    console.log(outerVar); // The returned inner function forms
                           // a closure over outerVar
  };
}

const closureFunc = outer(); // outer() returns the inner function
closureFunc(); // Output: I am from outer scope
```

In this example, the `inner` function forms a closure over the `outerVar` variable. Even though `outer()` has finished executing, `closureFunc` (which is `inner`) still has access to `outerVar`, demonstrating the concept of a closure.

Relationship between Lexical Scope and Closures

1. **Lexical Scope**: Functions can access variables from their parent scope due to where they are defined in the code.
2. **Closure**: When a function is returned or passed around, it "remembers" the variables from the scope it was defined in, allowing it to access those variables later.

Closures build upon lexical scope, allowing functions to access variables from their outer scope even after that scope is no longer active.

Example Combining Lexical Scope and Closures:

javascript Copy code

```javascript
function createCounter() {
  let count = 0; // This variable is in the lexical scope of createCounter

  return function increment() {
    count++; // The inner function forms a closure over the 'count' variable
    console.log(count);
  };
}

const counter1 = createCounter(); // Returns the 'increment' function with a closure over
counter1(); // Output: 1
counter1(); // Output: 2

const counter2 = createCounter(); // Returns a new 'increment' function with a separate cl
counter2(); // Output: 1 (new count for this closure)
```

Explanation:

- createCounter defines a variable count in its lexical scope.
- The increment function is returned from createCounter, forming a closure over count.
- Even after createCounter finishes executing, the returned increment function retains access to the count variable.
- Each call to createCounter creates a new closure with its own count variable, demonstrating how closures work with lexical scope.
-

Summary:

- **Lexical Scope** dictates what variables a function can access based on where it is defined.
- **Closures** extend lexical scope by allowing functions to "remember" variables from their outer scopes, even after the outer function has finished execution.

~ END ~